

Práctica REST MySQL e IoT

Natalia Hernández Piedrahita

Miguel Ángel Diuza

Juan Ospina Tenorio

Ejecución: 24 de septiembre de 2026 · America/Bogota.

Entorno: Ubuntu 22.04 ARM64, Vagrant y VirtualBox.

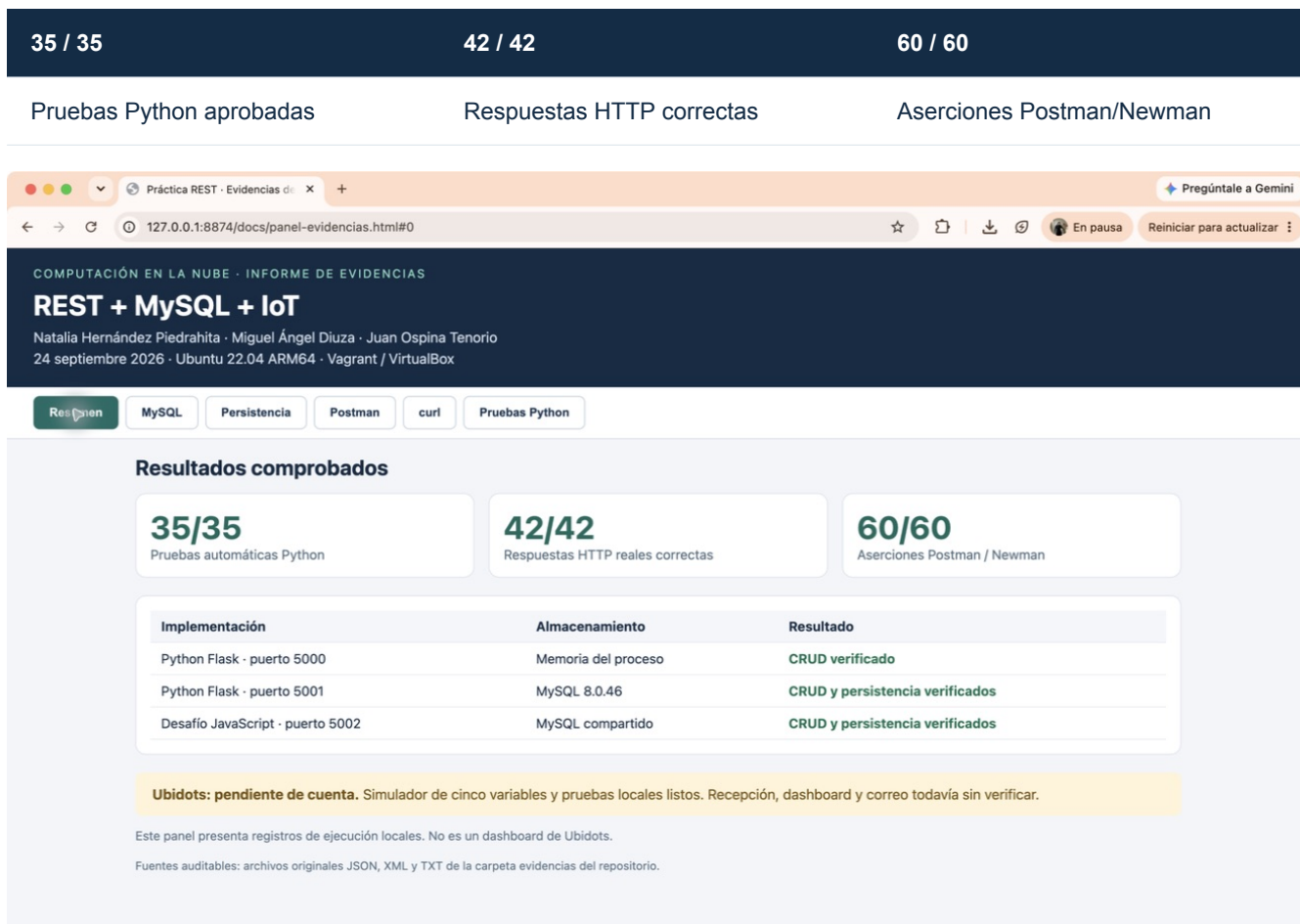


Figura 1. Captura real del panel local construido a partir de los registros de ejecución incluidos en el repositorio.

Estado: APIs de memoria y MySQL, desafío JavaScript y persistencia comprobados. El simulador Ubidots está implementado y probado localmente; faltan cuenta, recepción real, dashboard y alerta por correo.

Repositorio público: <https://github.com/nathernandez1189/practica-rest-mysql-ubidots>

API REST y ejecución sobre Ubuntu

La práctica implementa el recurso **books** con los campos id, title, description y author. Las dos APIs Flask comparten el contrato HTTP; la implementación JavaScript utiliza el mismo esquema MySQL.

Servicio	Almacenamiento	Puerto
Python / apirest.py	Memoria del proceso	5000
Python / apirest_mysql.py	MySQL 8.0.46	5001
JavaScript / server.js	MySQL 8.0.46	5002

Operación	Ruta	Resultado esperado
Consultar	GET /books y /books/<id>	200; 404 si no existe
Crear	POST /books	201 y cabecera Location
Actualizar	PUT /books/<id>	200; validación 400
Eliminar	DELETE /books/<id>	200; 404 si no existe

Se rechazan títulos vacíos, tipos incorrectos y campos desconocidos. Las consultas SQL se parametrizan; se comprobó que una cadena con sintaxis SQL se conserva como texto. Los errores de conexión se devuelven como 503 sin exponer datos internos.

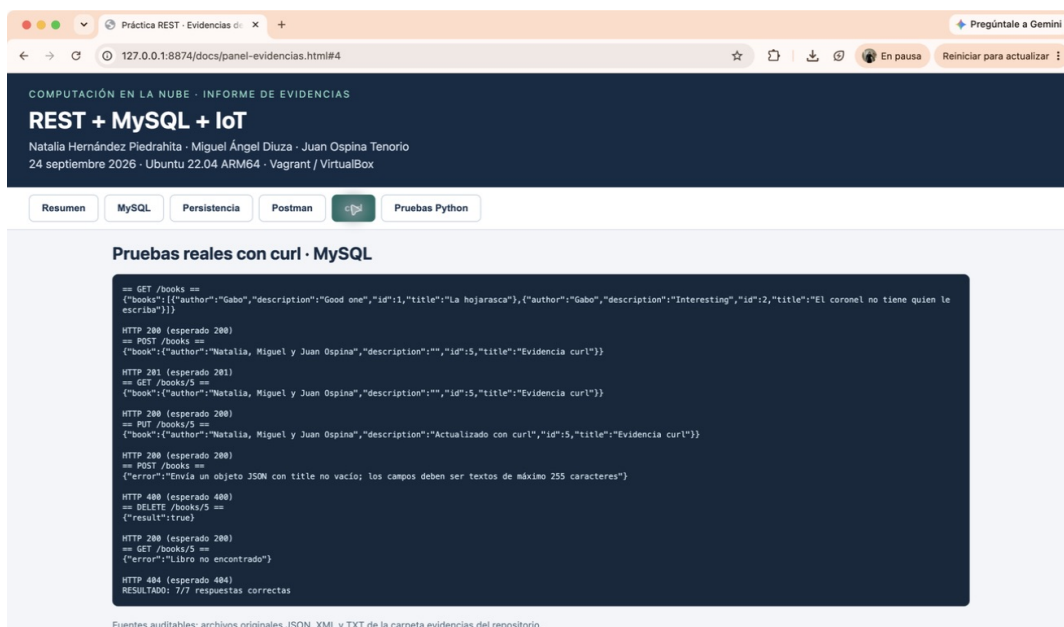


Figura 2. Registro real de curl contra MySQL: siete respuestas esperadas. También se ejecutó el mismo flujo en memoria y JavaScript.

Las APIs se instalaron en /home/vagrant/practica-rest-equip, sobre servidorUbuntu (192.168.100.3). La alternativa servidorRest del Vagrantfile fue validada como configuración; no se creó esa nueva VM.

MySQL y persistencia comprobada

La base exclusiva **rest_equipo** usa una tabla **books** con motor InnoDB y codificación utf8mb4. id es clave primaria autoincremental; title, description y author son VARCHAR(255). La cuenta de la API tiene permisos de lectura y escritura sobre esta base.

Origen	ID de prueba	Tras encender	Interpretación
memory	6	404	Se perdió en memoria
mysql	9	200	Se conservó íntegro
javascript	10	200	Se conservó íntegro

COMPUTACIÓN EN LA NUBE · INFORME DE EVIDENCIAS

REST + MySQL + IoT

Natalia Hernández Piedrahita · Miguel Ángel Dizuza · Juan Ospina Tenorio
24 septiembre 2026 · Ubuntu 22.04 ARM64 · Vagrant / VirtualBox

Resumen

MySQL

Persistencia

Postman

curl

Pruebas Python

Persistencia después de apagar y encender Ubuntu

Implementación	ID	Antes	Después	Verificación
memory	6	Creado (201)	404	Correcto
mysql	9	Creado (201)	200	Correcto
javascript	10	Creado (201)	200	Correcto

Memoria: el registro desaparece (404). **MySQL:** ambos registros sobreviven íntegros (200).

Current machine states:
servidorUbuntu poweroff (virtualbox)
The VM is powered off. To restart the VM, simply run 'vagrant up'

Antes: 2026-09-25T01:31:28.872574+00:00
Después: 2026-09-25T01:34:11.810735+00:00

Se apagó la VM de forma ordenada. No se borró el disco ni se destruyó la máquina.

Fuentes auditable: archivos originales JSON, XML y TXT de la carpeta evidencias del repositorio.

Figura 3. Resultados contrastados antes y después de un apagado real de Ubuntu. El archivo vm-apagada.txt registra el estado poweroff.

Los registros se crearon antes del apagado y se consultaron después del encendido. Se compararon sus campos completos. MySQL mantuvo los libros creados por Python y JavaScript; el registro de memoria devolvió 404.

Antes: 2026-09-25T01:31:28.872574+00:00
Después: 2026-09-25T01:34:11.810735+00:00

Respuesta al ejercicio: apagar la VM conserva los datos confirmados de MySQL en su disco. La memoria del proceso se reinicia. Destruir la VM o borrar su disco es diferente de apagarla y puede eliminar los datos.

Evidencias: persistencia.json, vm-apagada.txt, vm-encendido.txt, mysql-despues-encendido.txt y respaldo-rest_equipo.sql. La copia SQL contiene únicamente libros de esta práctica.

Pruebas reproducibles y Postman

Las comprobaciones se separan por herramienta. Algunas validan el mismo comportamiento desde niveles distintos; no se suman como si fueran requisitos únicos.

Grupo	Resultado	Archivo auditable
Python / pytest	35 aprobadas	evidencias/pytest.txt y pytest.xml
HTTP real	14 por API; 42 en total	evidencias/http-real.json
Postman / Newman	20 por API; 60 aserciones	evidencias/newman-5000/5001/5002.*
curl	7 por API; 21 respuestas	evidencias/curl-*.txt

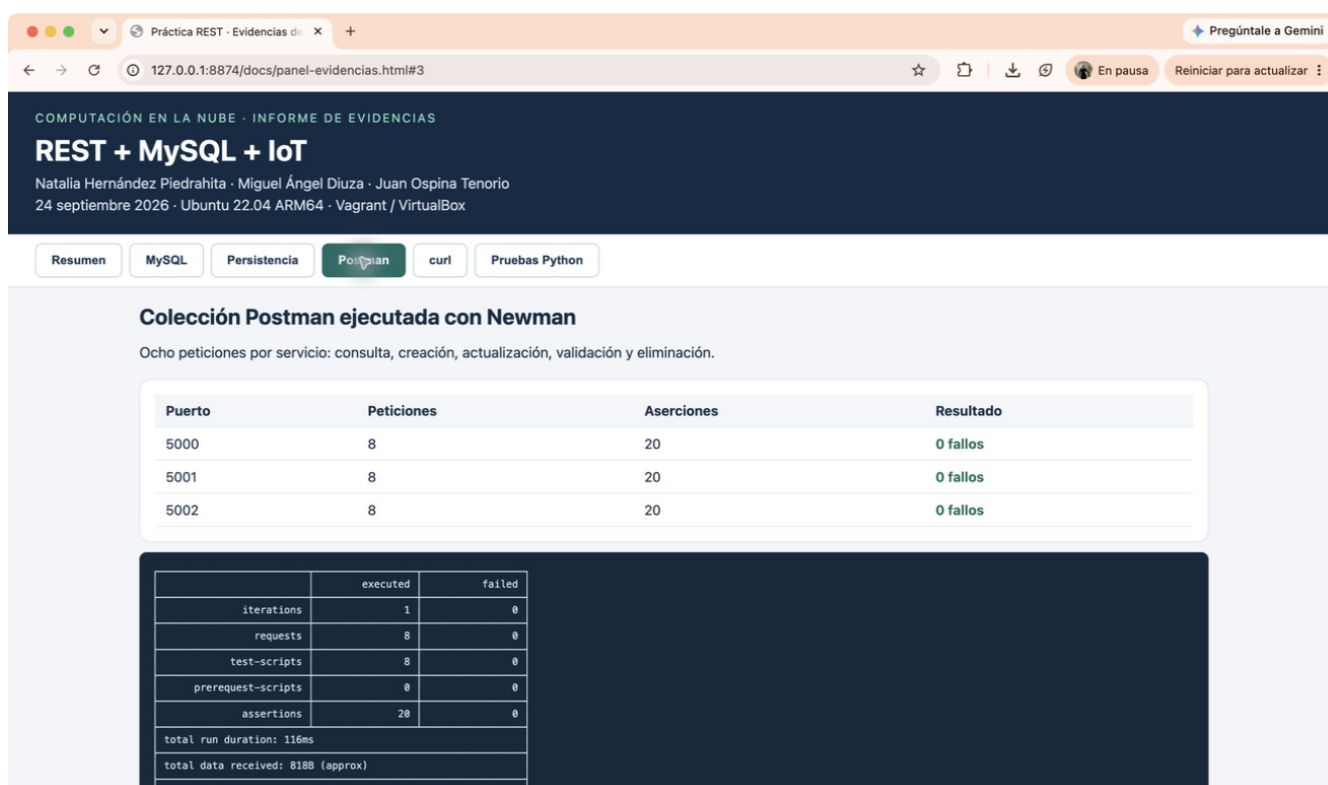


Figura 4. Captura del resumen de Newman: ocho peticiones y veinte aserciones por implementación, sin fallos.

Alcance de Postman: la colección se ejecutó con Newman, el ejecutor oficial de Postman. No se afirma haber utilizado la interfaz gráfica. La colección está lista para importarla y ejecutar Collection Runner.

Cobertura: CRUD completo, lectura de un recurso eliminado, ids inexistentes, títulos inválidos, campos inesperados, caracteres Unicode y texto con sintaxis SQL. Las pruebas Python agregan vaciado total de memoria, ids no reutilizados y respuestas 503 controladas.

La automatización de GitHub repite pruebas Python y pruebas de integración con un servicio MySQL real. El estado vigente debe consultarse en la pestaña Actions del repositorio; los archivos locales conservan el resultado de esta ejecución.

Ubidots: implementación y pendiente real

Esta parte aún no está completada en la nube. Durante la ejecución no había cuenta Ubidots disponible. Por eso no se certifica recepción de datos, dashboard ni correo. La simulación local y los dobles de prueba no sustituyen esas evidencias.

Variable	Contenido	Widget previsto
temperature	Temperatura simulada, 18 a 35 °C	Barras
humidity	Humedad simulada, 35 a 85 %	Gauge / indicador
position	Latitud y longitud ficticias	Mapa
luminosity	100 a 1000 lux; variable adicional	Indicador numérico
pressure	745 a 755 hPa; variable adicional	Línea

El cliente usa requests y HTTPS. El token se lee de UBIDOTS_TOKEN; no se guarda en el código. El envío usa una etiqueta de dispositivo validada, un tiempo máximo de espera y comprobación de errores internos por variable. El modo --dry-run muestra expresamente que no envía datos.

Pasos necesarios para cerrar el requisito

1. Crear y verificar una cuenta educativa Ubidots STEM y configurar el token local.
2. Enviar cinco muestras desde Python; repetir la actualización con curl y la colección Postman.
3. Crear los cinco widgets y capturar cambios de valores y marcas de tiempo.
4. Configurar temperature > 30 y un destinatario autorizado; enviar 24 y luego 34, y verificar el correo recibido.
5. Guardar las respuestas, el dashboard, el evento y el correo sin exponer credenciales.

La guía detallada está en docs/UBIDOTS.md. Las coordenadas son simuladas alrededor de Bogotá y no representan la ubicación de los integrantes. Los envíos son finitos para controlar el consumo de la cuenta.

Fuentes oficiales

ubidots.com/stem

Ubidots: simulación de datos en Python

Ubidots: condiciones y límites del plan STEM

Desafío, reproducción y sustentación

El desafío opcional se implementó en **JavaScript**, usando el módulo HTTP de Node.js y mysql2. Comparte el contrato de libros y la base con Python; se comprobó con las mismas pruebas HTTP, curl, Postman y persistencia.

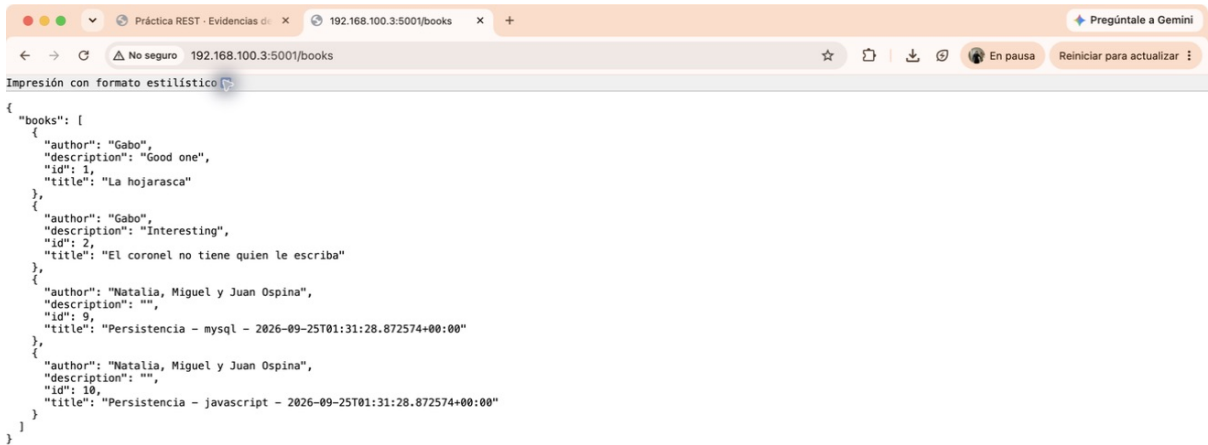


Figura 5. Consulta directa a la API activa en el navegador: libros iniciales y los dos marcadores conservados después del apagado.

Reproducir y revisar

1. Consultar README.md y docs/REPRODUCIR.md.
2. Encender servidorUbuntu desde ~/prueba con Vagrant; entrar por SSH.
3. En /home/vagrant/practica-rest-equipo ejecutar pytest y scripts/verify_http.py.
4. Importar las colecciones de postman/ o repetir Newman.
5. Revisar registros de evidencias/ y el informe; cerrar Ubidots cuando exista cuenta.

Integrante	Propuesta de explicación
Natalia Hernández Piedrahita	API REST, métodos HTTP y pruebas
Miguel Ángel Diuza	MySQL, consultas y persistencia
Juan Ospina Tenorio	Simulador IoT y desafío JavaScript

Este reparto es una propuesta para la sustentación, no una atribución de autoría del código. La guía incluye preguntas probables y respuestas técnicas.

Referencias docentes: guías REST (2024) y REST + MySQL (2023); repositorios de omondragon/APIRestFlask, APIRestFlaskMySQLUbuntu y UbidotsClient. Revisiones y adaptaciones documentadas en docs/REFERENCIAS.md. Los PDFs originales no se redistribuyen.

Entrega: el repositorio queda disponible para el profesor. Su publicación no equivale al envío en CampusDigital; no se ha realizado una entrega en ese portal desde esta tarea.